TCS IPA
Hard Coding Questions

Previous Year • 2023 – 2025 • Java & Python

35-Mark OOP Problems | 15-Mark Logic Problems | Full Solutions

10	35 + 15	OOP	Java+Python
Questions	Mark Split	Core Pattern	Both Solutions

Note: TCS IPA coding has 2 problems — a **35-mark OOP/class-based problem** and a **15-mark logic/algorithm problem**. Both are asked in the same 70-minute window. You can choose Java or Python. All questions below are based on real exam experiences reported by candidates (2023–2025).

Questions Index

PART 1 — 35-Mark OOP / Class-Based Problems

These are the hardest questions — always class design + method implementation

HARD Marks *Asked: Aug 2025*

Q1. Medicine Inventory — OOP Class + Case-Insensitive Search

Create a class **Medicine** with the following private attributes: **medicineId** (int), **medicineName** (String), **expiryYear** (int), **price** (double). Write necessary **getters and setters**. Create a class **Solution** and implement a static method **countMedicinesByMedicineName(Medicine[] arr, String medicineName)** that returns the count of medicines where medicineName matches (case-insensitive). In main(), read 4 Medicine objects and a search string, call the method, and print the count — or print **'No medicines found with that name'** if count is 0.

Sample Input / Output:

Input:	Output:
101 Paracetamol 2026 15.5 102 paracetamol 2025 12.0 103 Ibuprofen 2027 20.0 104 Aspirin 2024 8.0 PARACETAMOL	2

Approach / Logic: Loop through array, use equalsIgnoreCase() for name match, increment counter. Print count or fallback message.

Python Solution:

```
class Medicine:
def __init__(self, medicine_id, medicine_name, expiry_year, price):
self.__medicine_id = medicine_id
self.__medicine_name = medicine_name
self.__expiry_year = expiry_year
self.__price = price

def get_medicine_name(self):
return self.__medicine_name

class Solution:
@staticmethod
def countMedicinesByMedicineName(arr, medicine_name):
count = 0
for med in arr:
if med.get_medicine_name().lower() == medicine_name.lower():
count += 1
```

```
return count

medicines = []
for _ in range(4):
parts = input().split()
medicines.append(Medicine(int(parts[0]), parts[1], int(parts[2]), float(parts[3])))

search = input()
result = Solution.countMedicinesByMedicineName(medicines, search)
if result > 0:
print(result)
else:
print("No medicines found with that name")
```

HARD Marks *Asked: 2024*

Q2. Employee Salary Management — Inheritance + Method Override

Create a base class **Employee** with private attributes: **empId** (int), **empName** (String), **baseSalary** (double). Add getters/setters and a method **calculateBonus()** that returns **10% of baseSalary**. Create subclass **Manager** with extra attribute **teamSize** (int). Override **calculateBonus()** — Manager bonus = **10% of baseSalary + 500 * teamSize**. In **Solution** class, implement static method **getHighestBonus(Employee[] arr)** that returns the **empName** of the employee with the highest bonus. Read 3 Employee objects and 2 Manager objects in main(). Print the result.

Input:	Output:
E001 Alice 50000 E002 Bob 60000 E003 Carol 45000 M001 Dave 55000 5 M002 Eve 52000 8	Eve

Approach / Logic: Use polymorphism — store both Employee and Manager in Employee[] array. Override calculateBonus(). Loop to find max.

Python Solution:

```
class Employee:
def __init__(self, emp_id, emp_name, base_salary):
self._emp_id = emp_id
self._emp_name = emp_name
self._base_salary = float(base_salary)

def get_emp_name(self): return self._emp_name
def calculateBonus(self): return self._base_salary * 0.10

class Manager(Employee):
```

```
def __init__(self, emp_id, emp_name, base_salary, team_size):
super().__init__(emp_id, emp_name, base_salary)
self.__team_size = int(team_size)

def calculateBonus(self):
return self._base_salary * 0.10 + 500 * self.__team_size

class Solution:
@staticmethod
def getHighestBonus(arr):
return max(arr, key=lambda e: e.calculateBonus()).get_emp_name()

employees = []
for _ in range(3):
p = input().split()
employees.append(Employee(p[0], p[1], p[2]))
for _ in range(2):
p = input().split()
employees.append(Manager(p[0], p[1], p[2], p[3]))

print(Solution.getHighestBonus(employees))
```

HARD Marks *Asked: 2024*

Q3. Student Grade System — OOP + Sort by Marks Descending

Create class **Student** with private attributes: **studentId** (int), **studentName** (String), **marks** (double), **subject** (String). Add getters/setters and a method **getGrade()**: marks >= 90 → 'A', >= 75 → 'B', >= 60 → 'C', >= 50 → 'D', else 'F'. In **Solution**, implement **getTopStudentsBySubject(Student[] arr, String subject)** that returns a list of student names (sorted by marks descending) for the given subject. Read 5 students. Print names one per line, or **'No students found'** if empty.

Input:	Output:
1 Alice 88 Math 2 Bob 92 Science 3 Carol 76 Math 4 Dave 95 Math 5 Eve 81 Science Math	Dave Alice Carol

Approach / Logic: Filter by subject, sort by marks descending, print names. Use sorted() with key=lambda.

Python Solution:

```
class Student:
def __init__(self, sid, name, marks, subject):
self.__sid = int(sid)
self.__name = name
self.__marks = float(marks)
self.__subject = subject

def get_name(self): return self.__name
def get_marks(self): return self.__marks
def get_subject(self): return self.__subject

def getGrade(self):
if self.__marks >= 90: return 'A'
elif self.__marks >= 75: return 'B'
elif self.__marks >= 60: return 'C'
elif self.__marks >= 50: return 'D'
else: return 'F'

class Solution:
@staticmethod
def getTopStudentsBySubject(arr, subject):
filtered = [s for s in arr if s.get_subject().lower() == subject.lower()]
return sorted(filtered, key=lambda s: s.get_marks(), reverse=True)

```

```
students = []
for _ in range(5):
p = input().split()
students.append(Student(p[0], p[1], p[2], p[3]))

subject = input()
result = Solution.getTopStudentsBySubject(students, subject)
if result:
for s in result:
print(s.get_name())
else:
print("No students found")
```

HARD Marks *Asked: 2023*

Q4. Library Book Management — OOP + Filter by Author

Create class **Book** with private attributes: **bookId** (int), **title** (String), **author** (String), **price** (double), **available** (boolean). Add getters/setters. In **Solution**, implement static method **getAvailableBooksByAuthor(Book[] arr, String author)** that returns the **titles** of all available books (available=true) by that author (case-insensitive), sorted alphabetically. Read 5 books and author name. Print titles one per line, or **'No books available'**.

Input:	Output:
1 CleanCode RobertMartin 45.0 true 2 Refactoring FowlerMartin 55.0 false 3 DesignPatterns GoF 60.0 true 4 WorkingCode RobertMartin 40.0 true 5 AgileCode RobertMartin 50.0 false RobertMartin	CleanCode WorkingCode

Approach / Logic: Filter: author matches (case-insensitive) AND available==True. Sort alphabetically by title.

Python Solution:

```
class Book:
def __init__(self, bid, title, author, price, available):
self.__bid = int(bid)
self.__title = title
self.__author = author
self.__price = float(price)
self.__available = available.lower() == 'true'

def get_title(self): return self.__title
def get_author(self): return self.__author
def is_available(self): return self.__available
```

```
class Solution:
@staticmethod
def getAvailableBooksByAuthor(arr, author):
result = [b.get_title() for b in arr
if b.get_author().lower() == author.lower() and b.is_available()]
return sorted(result)

books = []
for _ in range(5):
p = input().split()
books.append(Book(p[0], p[1], p[2], p[3], p[4]))

author = input()
result = Solution.getAvailableBooksByAuthor(books, author)
if result:
for t in result: print(t)
else:
print("No books available")
```

HARD Marks *Asked: 2023*

Q5. Bank Account — OOP + Transaction History + Balance

Create class **BankAccount** with private attributes: **accountId** (int), **holderName** (String), **balance** (double), **transactions** (list of doubles). Add methods: **deposit(amount)** — add to balance and transactions list; **withdraw(amount)** — deduct if sufficient, else print 'Insufficient balance'; **getBalance()**; **getTransactionCount()**. In **Solution**, implement **getHighestBalanceAccount(BankAccount[] arr)** returning the holderName of the account with highest balance. Read 3 accounts with initial balance, then process transactions (D/W amount), print result.

Input:	Output:

`1 Alice 5000`
`2 Bob 3000`
`3 Carol 8000`
`D 1 2000`
`W 2 500`
`D 3 1000`
`END`
	`Carol`
	`3`
	`9000.0`

Approach / Logic: Use a dict to map accountId to object. Process D/W transactions. Find max balance with max(). Print name, tx count, balance.

Python Solution:

```
class BankAccount:
def __init__(self, acc_id, holder, balance):
self.__acc_id = int(acc_id)
self.__holder = holder
self.__balance = float(balance)
self.__transactions = []

def get_id(self): return self.__acc_id
def get_holder(self): return self.__holder
def get_balance(self): return self.__balance
def getTransactionCount(self): return len(self.__transactions)

def deposit(self, amount):
self.__balance += amount
self.__transactions.append(amount)

def withdraw(self, amount):
if amount <= self.__balance:
self.__balance -= amount
self.__transactions.append(-amount)
else:
print("Insufficient balance")
```

```
accounts = {}
for _ in range(3):
p = input().split()
acc = BankAccount(p[0], p[1], p[2])
accounts[int(p[0])] = acc

while True:
line = input()
if line == "END": break
p = line.split()
acc = accounts[int(p[1])]
if p[0] == 'D': acc.deposit(float(p[2]))
else: acc.withdraw(float(p[2]))

best = max(accounts.values(), key=lambda a: a.get_balance())
print(best.get_holder())
print(best.getTransactionCount())
print(best.get_balance())
```

PART 2 — 15-Mark Logic / Algorithm Problems

Shorter but tricky — test DSA and string skills

ARD Marks *Asked: 2025*

Q6. Anagram Check for Two Strings

Given two strings S1 and S2 (may contain spaces), determine if they are anagrams of each other (ignore case and spaces). Print **'Anagram'** or **'Not Anagram'**.

Input:	Output:
`Listen` `Silent`	`Anagram`

Approach / Logic: Remove spaces, convert to lowercase, sort both strings and compare. Or use Counter/frequency map.

Python Solution:

```
s1 = input().replace(" ", "").lower()
s2 = input().replace(" ", "").lower()
print("Anagram" if sorted(s1) == sorted(s2) else "Not Anagram")
```

HARD Marks *Asked: 2024*

Q7. Longest Consecutive Subsequence in an Array

Given an array of N integers, find the length of the longest consecutive elements sequence. Elements need not be contiguous in the array. E.g. [100, 4, 200, 1, 3, 2] → longest consecutive sequence is [1,2,3,4] → length 4.

Input:	Output:
`6` `100 4 200 1 3 2`	`4`

Approach / Logic: Convert array to set. For each element, check if it's the start of a sequence (num-1 not in set). Count forward. O(n) time.

Python Solution:

```
n = int(input())
nums = list(map(int, input().split()))
num_set = set(nums)
max_len = 0

for num in num_set:
if num - 1 not in num_set: # start of sequence
curr = num
```

```
length = 1
while curr + 1 in num_set:
curr += 1
length += 1
max_len = max(max_len, length)

print(max_len)
```

HARD Marks *Asked: 2024*

Q8. Matrix Spiral Print

Given an M x N matrix, print all elements in spiral order (clockwise from outside to inside).

Input:	Output:
3 3 1 2 3 4 5 6 7 8 9	1 2 3 6 9 8 7 4 5

Approach / Logic: Use 4 boundaries: top, bottom, left, right. Traverse right → down → left → up, shrinking boundaries each pass.

Python Solution:

```
m, n = map(int, input().split())
matrix = [list(map(int, input().split())) for _ in range(m)]

top, bottom, left, right = 0, m-1, 0, n-1
result = []

while top <= bottom and left <= right:
for i in range(left, right+1): result.append(matrix[top][i])
top += 1
for i in range(top, bottom+1): result.append(matrix[i][right])
right -= 1
if top <= bottom:
for i in range(right, left-1, -1): result.append(matrix[bottom][i])
bottom -= 1
if left <= right:
for i in range(bottom, top-1, -1): result.append(matrix[i][left])
left += 1

print(*result)
```

ARD Marks *Asked: 2023*

Q9. Balanced Parentheses Check

Given a string containing only the characters **'(' ')' '{' '}' '[' ']'**, determine if the input string is valid (balanced). Print **'Balanced'** or **'Not Balanced'**. Opening brackets must be closed by the same type in correct order.

Input:	Output:
{[()]}	Balanced

Input:	Output:
([)]	Not Balanced

Approach / Logic: Classic stack problem. Push open brackets. On close bracket, check if stack top matches. If empty at end → Balanced.

Python Solution:

```
s = input()
stack = []
mapping = {')': '(', '}': '{', ']': '['}

for ch in s:
if ch in '({[':
stack.append(ch)
elif ch in ')}]':
if not stack or stack[-1] != mapping[ch]:
print("Not Balanced")
exit()
stack.pop()

print("Balanced" if not stack else "Not Balanced")
```

HARD Marks *Asked: 2023*

Q10. Count Substrings with Equal Number of 0s and 1s

Given a binary string (containing only '0' and '1'), count the number of substrings that have an equal number of 0s and 1s. E.g. '0011' → substrings '01'(pos 1-2), '0011'(pos 0-3), '01'(pos 2-3) → Answer: 3

Input:	Output:
0011	3

Approach / Logic: Replace 0 with -1 and 1 with +1. Use prefix sum. If prefix_sum[i] == prefix_sum[j], substring [i+1..j] has equal counts. Use a hashmap to count prefix sum frequencies. Answer = sum of C(freq, 2) for each prefix sum value.

Python Solution:

```
from collections import defaultdict

s = input()
# Replace 0 -> -1, 1 -> +1
nums = [-1 if c == '0' else 1 for c in s]

prefix_count = defaultdict(int)
prefix_count[0] = 1 # empty prefix
prefix_sum = 0
count = 0

for num in nums:
prefix_sum += num
count += prefix_count[prefix_sum]
prefix_count[prefix_sum] += 1

print(count)
```

Quick Strategy for 70-Minute Coding Window:

1. Read BOTH problems first (2 min). Pick the 15M logic problem first if it looks easier.
2. For 35M OOP: Class + getters/setters = ~8 marks. Static method = ~15 marks. Main + I/O = ~12 marks.
3. Partial marks are given per test case — even a partially working OOP solution scores 15–20 marks.
4. Always handle edge cases: empty input, case-insensitive matching, zero count fallback messages.
5. For Python: use @staticmethod, list comprehensions, and sorted() with key=lambda.